

Societal Problem Database

Group 12

September 2026

1 Our Societal Problem

1.1 Scope

Our chosen Societal Problem is about the increasing number of cycling accidents in the Netherlands. We chose to investigate why this number is increasing. We want to build a database that can effectively handle nationwide accident registration and querying.

1.2 How the data interacts

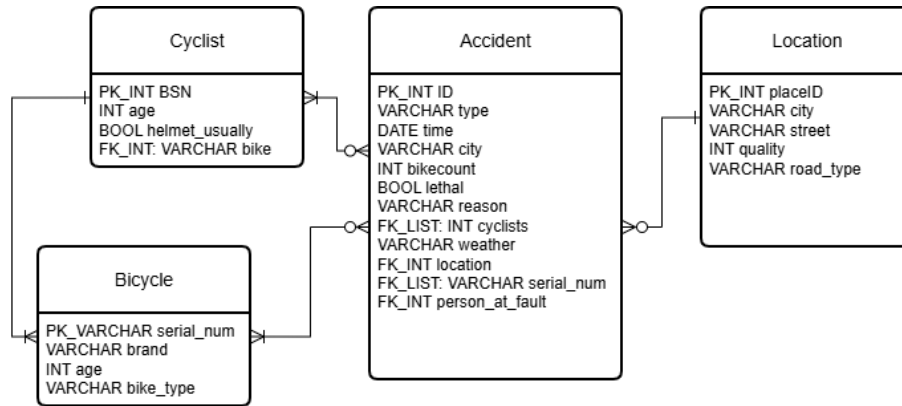
- A Cyclist person can have one or more bicycles. One bicycle can belong to only one cyclist while driving, we do not operate with shared-ownership bikes, and we assume that the bicycle owner uses their own bike.
- An Accident can be caused by one or more cyclists. One cyclist can cause zero or more accidents.
- One bicycle can have zero or more accidents. One accident can have one or more bicycles involved, because otherwise there would not be bicycle accident.
- One Location can have zero or more accidents. One accident can happen at only one location.

1.2.1 Entities:

- Cyclist: Represents one cyclist person.
- Bicycle: Represents one bicycle.
- Accident: Represents one accident by bicycle.
- Location: Represents one location, where an accident could happen.

1.2.2 Relationships

- Cyclist → have/own → Bicycle (One-to-many)
- Cyclist → cause → Accident (many-to-many)
- Bicycle → involved → Accident (many-to-many)
- Location → happen → Accident (one-to-many)



1.3 NF optimization

1.3.1 Entities and relations

- **Accident:** This entity provides information about a specific accident. The different influences are stored in Foreign Keys except for time. Time is an event based feature so this scales with the eventID.
- **Location:** The location of the accident, with a unique LocationID, city, street but also more specific road_quality_score. And finally, what Road-Type the accident occurred in.
- **RoadType:** A location is defined by a street. Streets differ very much from material, making for different situations. Clinkers and asphalt react different to weather influences. Thats the reason we chose to make a new table with all possible roadtypesID and give this roadType a name and rain_effect_score.
- **AccidentType:** Not all accidents are the same. To gain better insights on how the accident happened, the type of accident is recorded, and if the accident happend through bad weather by rain_effect_score. This is also used to limit the different types from any VARHCAR to a list of possibilities.
- **ReasonType:** As mentioned by previously stated research paper, bicycle accidents happen because of four different reasons. *Lack of alertness, lateral instability, lack of protective equipment and conspicuity aids* and finally *errors and violations*. To correctly categorize these reasons, a new table has been created.
- **DownfallType:** To correctly identify what kind of rain happened, the DownfallType table exists. These are categories again.
- **CyclistAccident:** To correctly identify who actually is involved in an accident, the CyclistAccident has been made. Every CyclistAccidentID is matched with a *FK_AccidentID*, so everybody involved is being traced correctly. In this table we track the *FK_CyclistID*, *leathal_status* and if the cyclist was *at_fault*.
- **Cyclist:** To know who is involved in the accident, a cyclist table has been created. It provides some more details on the riders behavior.
- **Bicycles:** To know what kind of bicycles are in accidents, a new table has been made. A cyclist could have more bycycles, so the *PK_serial_num* has been made. A few characterization have been noted in this table.
- **BicycleOwner:** Used to connect bicycles to their owners. Which bicycles does the cyclist own.

1.4 Changes with first database

To ensure our database can efficiently handle queries regarding cycling accidents without data anomalies, we applied the three fundamental database normalization forms. Below is a detailed breakdown of all changes made to the prior database architecture.

1.4.1 1NF: Atomic values

In our draft ERD, the Accident entity contained two attributes that violated 1NF: *FK_cyclists* and *FK_serial_num*. 1NF dictates that every cell must contain exactly one single, indivisible value. By storing lists of cyclists and bicycles in a single column, querying which specific cyclist was involved in an accident would offend this definition. Therefore a bridge entity CyclistAccident was created.

Accident				
ID	Type	...	cyclists	serial_nums
1	Fall	...	Cyclist_1	123456
2	Crash	...	Cyclist_2, Cyclist_3, Cyclist_7	345678, 314872, 382164
3	Mass Accident		Cyclist_6, Cyclist_8, Cyclist_11,	216372, 981263, 3612323,

Accident		Bridge		Cyclist	
ID	Type	AccidentID	CyclistID	ID	Name
1	Fall	1	1	1	Alan
2	Crash	2	2	2	Bob
		2	3	3	Camille
3	Mass Accident	2	5	4	Diana
				5	Edward

1.4.2 2NF: No partial dependencies

2NF requires that all non-key attributes depend on the entire primary key. Because our initial draft ERD utilized auto-incrementing Primary Keys rather than combined Foreign Keys, there were no partial dependencies in our core tables. In the new ERD, all entities with relationship 'many', has atleast one other foreign key.

1.4.3 3NF: No transitive dependencies

3NF requires that non-key columns must not depend on other non-key columns. The old ERD had multiple dependencies in the location entity. This would explode the location table, because each different combination of categories would get its own row. Now, by splitting up these tables even more, each column is independent.

Accident				
ID	Type	Location	City	Time
1	Fall	1	Maastricht	10:31
2	Crash	2	Maastricht	14:14
3	Mass Accident	3	Amsterdam	16:10

Location		
ID	City	Street
1	Maastricht	Grohte Graht
2	Maastricht	Paul-Henry Spraaklaan
3	Amsterdam	Cool Straat

Accident			
ID	Type	Location	Time
1	Fall	1	10:31
2	Crash	2	14:14
3	Mass Accident	3	16:10

Location		
ID	City	Street
1	Maastricht	Grohte Graht
2	Maastricht	Paul-Henry Spraaklaan
3	Amsterdam	Cool Straat

